

Final Report

DramaScope: Intelligent Video Scene and Narrative Analysis - Final Project

Author: Kim Tsadok

Supervisor: Eyal Eisenstein

Institution: Department of Computer Science and Software Engineering,
Afeka College of Engineering

Date: July 2026

Abstract

Artificial Intelligence (AI) has significantly advanced multimedia analysis, including computer vision and automated video understanding. However, existing tools often provide either low-level detections, such as objects and shot boundaries, or general scene descriptions, without combining both into one structured and consistent analysis.

This project aims to develop an automated video analytics pipeline for analysing short video scenes. The system uses Google Cloud Video Intelligence to extract shot changes, labels, objects and human presence and converts them into features such as shot frequency, object entropy, interaction density and human presence ratio.

These features are normalized and used to calculate a Scene Complexity Score and classify each video into a narrative phase, such as Calm, Dynamic, Dense, or Static. In parallel, representative frames are analysed by a Large Vision-Language Model to generate a scene summary and structured information about the setting, entities, actions, emotions and interaction level.

The final system combines quantitative video analysis with semantic interpretation and stores the results in structured JSON files for further analysis and future multimedia applications.

Table of Contents

1. Introduction	1-2
2. Background and Related Work.....	3-4
3. Methods and System Design	5-10
4. Methodology	11-19
5. Implementation	20-23
6. Experiments and Results	24-29
7. Discussion	30
8. Conclusion and Future Work.....	31
9. References	32
10. Credits.....	33

1. Introduction

- **1.1 Motivation**

My goal with this project is to develop an AI based tool with LVLM and API's that will analyze scenes and scenarios from movies, shows, etc.

Existing video-analysis tools often provide either technical detections, such as objects and shot boundaries, or general scene descriptions, but rarely combine both into a structured and consistent result.

The motivation of this project is to bridge this gap by developing an automated pipeline backed by research, that produces both quantitative scene features and meaningful semantic interpretation.

- **1.2 Problem Statement**

Can I analyze as accurately and as quickly as possible, a short video depicting a scene?

- **1.3 Project Goals**

- Develop an automated pipeline for analyzing short video clips.
- Use Google Cloud Video Intelligence to detect shots, objects, labels and human presence.
- Use a Large Vision-Language Model to generate a semantic scene summary.
- Present the gathered information from both layers in a concise and meaningful Json files.

- **1.4 Brief Overview**

My project addresses the challenge of automatically analysing short video scenes by combining visual feature extraction with semantic interpretation.

The system identifies structural and narrative characteristics of each video, including scene complexity, narrative phase, visible entities, actions, emotions and interaction level.

I developed an end-to-end pipeline that combines Google Cloud Video Intelligence, feature-engineering algorithms and a Large Vision-Language Model to produce structured and interpretable video metadata.

Main Contributions:

- **Research-Oriented Development**

The project was conducted as a research project, including extensive investigation of LVLM capabilities, limitations, prompting methods, structured outputs and their application to video understanding.

- **Academic Collaboration**

The research was supported through collaboration with Dr. Shmuel Tyszberowicz and his Chinese student, contributing additional knowledge and guidance regarding LVLM technologies and current research directions.

- **Scene-Level Feature Extraction**

The system derives measurable features such as shot frequency, object entropy, interaction density and human presence ratio.

- **Structured Scene Interpretation**

The pipeline calculates scene complexity, classifies narrative phase, identifies interaction level and generates structured metadata describing entities, actions, emotions and scene context.

- **Reliable and Reusable Output**

The results are parsed, validated and stored in structured JSON files for evaluation and future multimedia-analysis applications.

- **Narrative Phase Classification**

A rule-based classifier assigns each video a narrative phase: Calm, Dynamic, Dense, or Static. The system also records the reasons behind the selected classification, improving interpretability.

- **Large Vision-Language Model Integration**

Representative frames are analysed by an LVLM to produce semantic information that traditional detection APIs do not provide directly, including:

- scene setting
- main entities
- visible actions
- emotional indicators
- interaction level
- visual evidence
- scene summary

2. Background and Related Work

- **2.1 Related Work**

Recent research in computer vision and multimodal AI has shown that Vision-Language Models can combine visual information with natural-language reasoning to produce semantic descriptions and structured outputs.

The ColPali study demonstrates how visual content can be represented directly by a Vision-Language Model and highlights the importance of multimodal understanding, efficiency and interpretability. Although it focuses on document retrieval, its principles are relevant to this project's use of LVLM for structured video understanding.

The study about Video Intelligence API by IEEE shows that small, carefully inserted visual changes can mislead the API's video labels.

This is relevant to My project because it highlights that Google Cloud outputs should not be treated as absolute ground truth. It supports My decision to validate the extracted metadata and combine it with an LVLM-based semantic layer for more reliable and interpretable video analysis and shows the power and motivation behind choosing GCP's Video Intelligence API to use in My project.

This project therefore combines two complementary approaches: deterministic video metadata extraction through Google Cloud Video Intelligence and higher-level semantic interpretation through a Large Vision-Language Model (LVLM).

The main technologies include Python, OpenCV, Google Cloud services, GLM vision models and structured JSON outputs.

The datasets for this project are as follows: Dramatic scenes from Kaggle.com (up to 30 secs)

- **2.2 Existing Approaches**

Existing video-analysis solutions provide only partial support for the project's goals or rely on single type of analysis:

- **ChatGPT and similar multimodal assistants** can describe scenes and visible behaviour, but their video support and available analysis tools may vary between models and environments.
- **Perplexity, Claude, DeepSeek and Grok** were unable to directly analyse the uploaded video and required extracted frames, screenshots, or a written description instead.
- **Google Cloud Video Intelligence** reliably detects shot boundaries, labels, objects and confidence values, but mainly produces low-level metadata rather than a complete semantic explanation of the scene.
- **Standalone LVLMs such as GLM** can provide detailed scene summaries, actions, settings and interaction descriptions, but their results are interpretive and may require prompt tuning, parsing and validation.

My project combines these approaches into one structured pipeline.

Google Cloud provides measurable and deterministic video metadata, while the LVLM adds higher-level semantic understanding.

The resulting output is normalized, validated and stored in structured JSON, making the analysis more consistent, explainable and reusable than relying on a single existing tool.

2.3 Gaps Highlighted

The main gap addressed by this project is the lack of a unified system that combines measurable video metadata with high-level semantic scene understanding.

Existing tools usually provide either low-level detections, such as objects and shot boundaries, or general scene descriptions, but not both in a structured and consistent form.

The project bridges this gap by integrating Google Cloud Video Intelligence with an LVLM-based interpretation layer. It transforms raw detections into normalized features, scene complexity scores, narrative phase classifications and structured semantic metadata.

This creates a more explainable and reusable video-analysis output that can support researchers, filmmakers, students, even the visually impaired and future multimedia applications without relying on a single model or unstructured description.

3. Methods and System Design

- **3.1 Functional requirements:**

Video Input and Processing:

Allow the user to select a short video and process it through the full analysis pipeline.

Visual Metadata Extraction:

Use Google Cloud Video Intelligence to detect shot boundaries, labels, object tracks and human presence.

Feature Engineering:

Calculate the project's core scene features: shot frequency, object entropy, interaction density and human presence ratio.

LVLM Semantic Analysis:

Sample representative frames and use a Large Vision-Language Model to identify the setting, main entities, actions, emotional indicators, interaction level and scene summary.

Execution Modes and Outputs:

Support analysis of a single video, batch processing and separate GCP or LVLM tuning workflows and present the info gathered in structured JSON outputs like VideoFeatures.json and VideoInterpretation.json.

- **3.2 Non - Functional requirements:**

Performance:

The system should analyse short video clips within less than a minute per video - while minimizing unnecessary API calls and repeated frame extraction.

Reliability:

The pipeline should handle temporary API failures, invalid responses and missing data through retries, validation and fallback values.

Scalability:

The system should support both single-video analysis and batch processing for larger evaluation sets.

Maintainability:

The project should use a modular architecture in which cloud analysis, feature engineering, LVLM processing, validation and orchestration are separated into dedicated components.

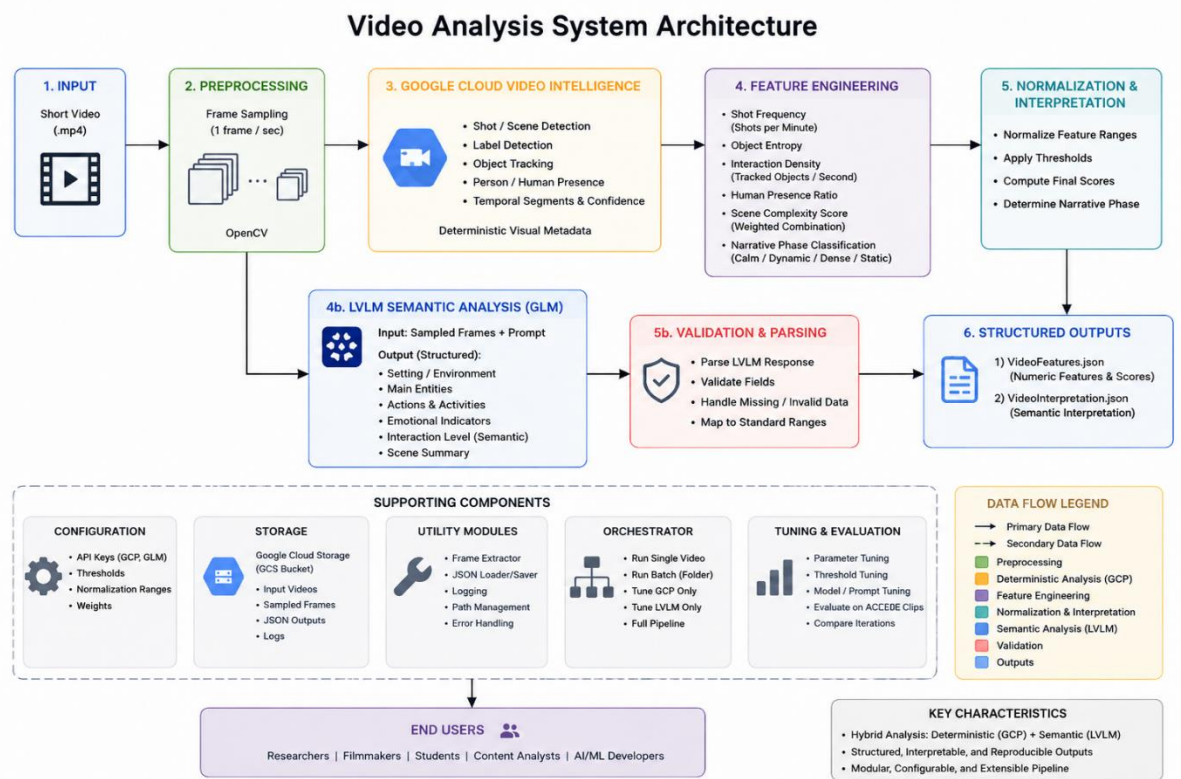
Usability:

The system should provide clear outputs that can be understood by both technical and non-technical users.

Extensibility:

The architecture should allow future improvements/updates of the LVLM, adjustment of thresholds, addition of new features and integration with a web interface.

- 3.3 System architecture:



My system presents a hybrid video-analysis pipeline in which Google Cloud Video Intelligence extracts measurable visual metadata, while an LVLM produces higher-level semantic interpretation.

These outputs are normalized, validated and combined into structured JSON files containing scene features, complexity scores, narrative phase and semantic metadata.

GLM Module

The GLM module performs the semantic analysis of the video.

It receives sampled frames and versioned prompts, then generates both a free-text scene summary and structured metadata such as the setting, main entities, actions, emotional indicators, interaction level and interaction evidence. Its responses are parsed and validated before being added to the final interpretation output.

GCP API Module

The Google Cloud Video Intelligence module extracts deterministic visual metadata from the uploaded video. It detects shot boundaries, labels, object tracks, confidence values and human presence over time. These results are stored in VideoFeatures.json and serve as the input for the algorithm layer.

Algorithm Module

The algorithm module converts the GCP metadata into interpretable scene-level features. It calculates shot frequency, object entropy, interaction density and human presence ratio, normalizes them and combines them into a Scene Complexity Score. It also applies rule-based thresholds to classify the scene as Calm, Dynamic, Dense, or Static.

Backend Pipeline Module

The backend pipeline coordinates the complete system workflow. It handles video input, Google Cloud Storage upload, GCP analysis, frame extraction, GLM inference, parsing, validation, feature calculation and JSON output generation. It also supports single-video processing, batch execution, GCP-only tuning, LVLM-only tuning, retries, fallbacks and error handling.

- **3.4 Technology stack:**

My technology stack is centered on Python, using OpenCV, Google Cloud Storage, Google Cloud Video Intelligence and GLM vision models to process, analyze and interpret short videos. GitHub, JSON outputs, environment variables and Markdown evaluation files support version control, configuration, reproducibility and experimentation.

Backend

- **Python** – Core language used for the video-analysis pipeline, feature engineering, validation and orchestration.
- **Pipeline modules** – Manage video input, GCS upload, GCP analysis, LVLM inference, parsing, validation, tuning modes and JSON output generation.

Libraries

- **OpenCV** – Extracts and samples video frames for LVLM analysis.
- **Google Cloud Python libraries** – Connect the project to Google Cloud Storage and the Video Intelligence API.
- **OpenAI-compatible client library** – Sends prompts and sampled frames to the GLM vision model.
- **NumPy** – Supports numerical calculations and feature processing.
- **Standard Python libraries** – Used for JSON handling, file paths, logging, configuration, retries and error handling.

APIs and AI Models

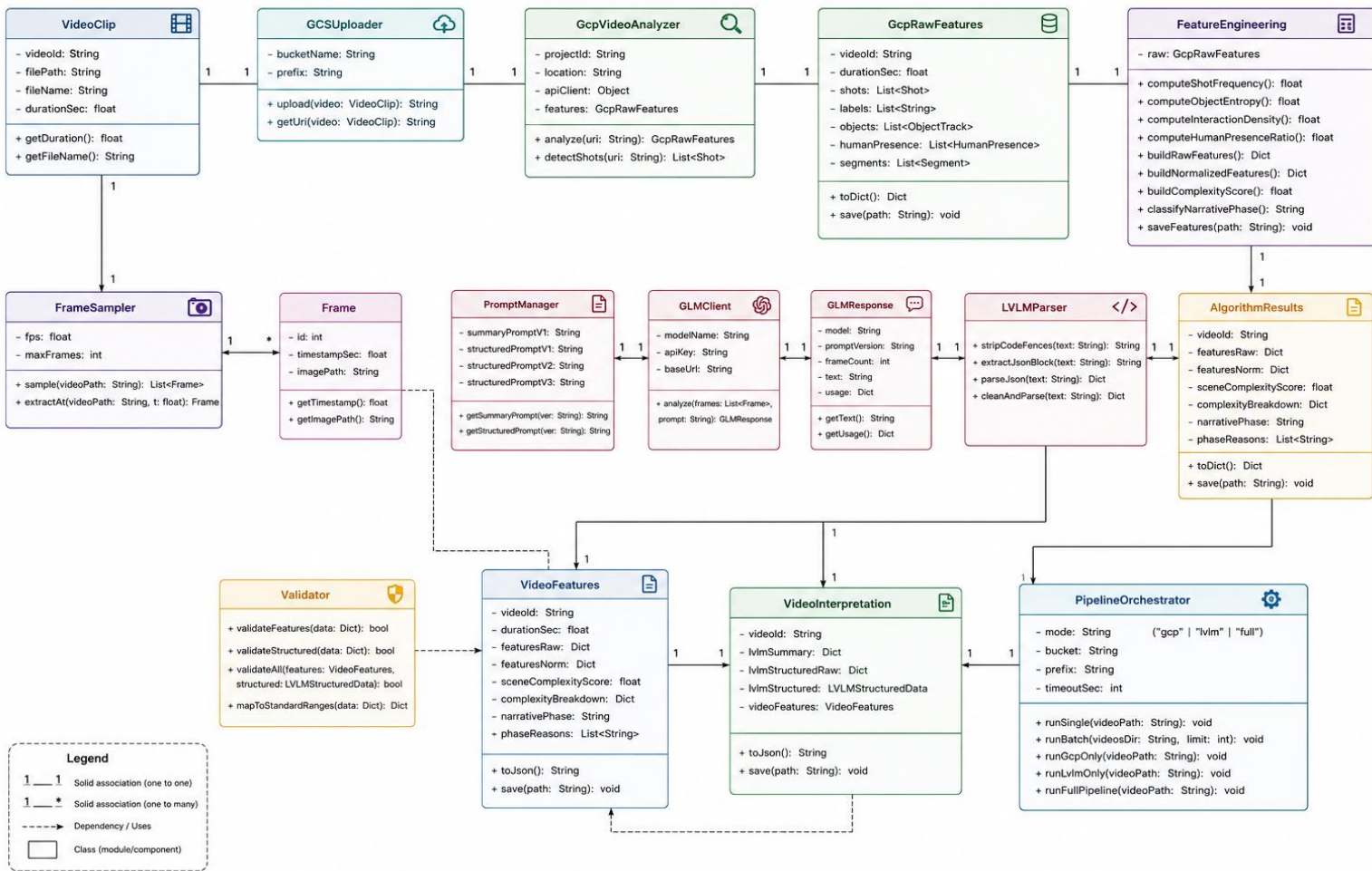
- **Google Cloud Video Intelligence API** – Detects shot boundaries, labels, object tracks, confidence values and person-related metadata.
- **Google Cloud Storage** – Stores uploaded videos before cloud analysis (bucket).
- **GLM-4.6V-Flash** – Used during the initial LVLM integration and prompt-tuning stages.
- **GLM-5V-Turbo** – Used in later experiments for improved video understanding and structured semantic output.
- **Other evaluated models and platforms** – ChatGPT, Gemini, Claude, DeepSeek, Perplexity, Grok, Qwen-VL and LLaVA-related models were reviewed or tested as part of the project's Competitor's Review.

Additional Tools

- **Git and GitHub** – Version control, experimentation history, documentation and code management.
- **JSON** – Main structured format for VideoFeatures.json and VideoInterpretation.json.
- **Environment variables** – Secure storage of GCP and GLM API credentials. (.env)
- **Markdown evaluation files** – Used to document feature statistics, prompt iterations, expected-versus-predicted results and tuning conclusions.

Class Diagram

Short Video Scene Analysis System – Class Diagram



4. Methodology

- 4.1 Dataset(s)

The project uses one primary dataset: the **ACCEDE video dataset**.

Source

ACCEDE, or the *Affective Video Content Database*, contains short video clips extracted from Creative Commons films.

It was originally developed for research in affective computing and multimedia analysis. The project uses these clips to evaluate both numeric scene analysis and LVLM-based semantic interpretation.

Structure

Video format: .mp4

Typical duration: approximately 8–12 seconds per clip

Content: short scenes containing people, objects, environments, actions and different editing patterns

External metadata: affective annotations, including arousal and valence information, are available with the original dataset, but they are not used as direct inputs to the current pipeline.

Size

The local dataset contains **570 short video clips**. A smaller subset is used during development, threshold tuning, prompt engineering, regression testing and final evaluation. (up to 40~ videos)

Preprocessing

Each selected video is uploaded to Google Cloud Storage and analysed by Google Cloud Video Intelligence to extract duration, shot boundaries, labels, object tracks, confidence scores and human-presence intervals.

The pipeline then calculates shot frequency, object entropy, interaction density and human presence ratio. Representative frames are also sampled at approximately one frame per second for LVLM analysis.

Evaluation Data

In this project, subsets of the dataset are manually reviewed to define expected narrative phases and interaction levels, as well as feature stats which will be further reviewed.

These expected values are compared with the pipeline results during normalization tuning, threshold adjustment, model comparison and prompt refinement.

- **4.2 Algorithms and models used:**

My system combines deterministic feature engineering, rule-based classification and Large Vision-Language Model analysis to produce structured interpretations of short video scenes.

- **Feature Extraction Algorithms**

Shot Frequency

Calculated as the number of detected shots divided by the video duration.
It represents the editing pace of the scene.

Object Entropy

Calculated using Shannon entropy over the object frequency distribution.
It measures the diversity of detected object types and serves as an indicator of visual complexity.

Interaction Density

Calculated as the number of tracked object segments divided by the video duration.
It represents scene busyness rather than direct social interaction.

Human Presence Ratio

Calculated as the total merged duration in which people are detected, divided by the full video duration. It indicates how strongly the scene is centered on human subjects.

- **Scene Complexity Algorithm**

The features are normalized and combined using a weighted formula to calculate a continuous **Scene Complexity Score**.
The score provides an interpretable measure of the scene's structural and visual complexity.

Narrative Phase Classifier

A rule-based classifier uses the normalized features to assign one of the narrative phases:

- **Calm** – balanced or moderate scene
- **Dynamic** – fast-paced without high visual density
- **Dense** – visually crowded and complex
- **Static** – minimal activity and slow pacing

The classifier also records the conditions that caused the selected result, making the output plausible.

Google Cloud Video Intelligence

Google Cloud Video Intelligence is used as the deterministic computer-vision model.
It detects shot boundaries, labels, object tracks, confidence values and person-related temporal information, which are then converted into the project's feature engineering stats.

- **Large Vision-Language Models**

The project initially used GLM-4.6V-Flash and later evaluated GLM-5V-Turbo.

The LVLM analyses sampled frames and produces a scene summary and structured fields such as setting, main entities, actions, emotion words, interaction level and interaction evidence.

The LVLM interaction scale ranges from 0 to 3, from no visible interaction to strong or highly active interaction. Prompt engineering and model comparison were used to improve the consistency of these semantic predictions.

This project's core interpretation is based on engineered features, configurable thresholds and LVLM-generated semantic metadata.

Input Features

The algorithm receives my normalized video features:

- shot_frequency
- object_entropy
- interaction_density
- human_presence_ratio

The LVLM also provides structured semantic fields such as:

- setting
- main_entities
- actions
- emotion_words
- interaction_level
- interaction_evidence

Output

The system produces:

- a continuous scene_complexity_score
- a complexity_breakdown
- a narrative phase classification: Calm, Dynamic, Dense, or Static
- an LVLM-generated scene summary
- structured semantic metadata
- validated JSON outputs in VideoFeatures.json and VideoInterpretation.json.

- **4.3 Tools used**

My project integrates tools for cloud-based video analysis, frame processing, semantic interpretation, feature engineering, validation and pipeline execution.

Data Processing and Algorithms

- **Python** – Core programming language used throughout the project.
- **NumPy** – Supports numerical calculations, normalization, entropy computation and feature processing.
- **JSON** – Stores the structured outputs in VideoFeatures.json and VideoInterpretation.json.

Video Processing

- **OpenCV** – Used to read videos and extract representative frames for LVLM analysis.
- **Google Cloud Storage** – Stores uploaded video files before cloud-based processing.
- **Google Cloud Video Intelligence API** – Extracts shot boundaries, labels, object tracks, confidence values and human-presence information.

Large Vision-Language Model Processing

- **GLM-4.6V-Flash** – Used during the initial LVLM integration and prompt-tuning stages.
- **GLM-5V-Turbo** – Used in later experiments for semantic scene analysis and structured metadata generation (as a more compatible and better model).
- **OpenAI-compatible API client** – Sends frames and prompts to the GLM service.
- **Prompt parsing and validation tools** – Extract, validate and normalize the LVLM's structured JSON responses.

Pipeline and Development Tools

- **Python command-line pipeline** – Supports full analysis, GCP-only processing, GCP tuning, LVLM tuning, single-video execution and batch processing.
- **Git and GitHub** – Used for version control, code management, experimentation history and documentation.
- **Environment variables** – Store API keys and cloud credentials securely.
- **Markdown files** – Document feature statistics, evaluation tables, prompt versions, testing results and tuning conclusions.

- **4.4 Implementation steps**
or iterations (Agile, CRISP-DM, etc.)

My project followed an iterative and modular development process similar to Agile, with continuous research, implementation, testing, prompt tuning and refinement.

The main development stages were:

1. **Research and Planning**
I researched video-analysis APIs, multimodal models and LVLM technologies to define the project scope and architecture. I also collaborated with Dr. Shmuel Tyszberowicz's Chinese student, who advised us on LVLM model selection, keyframe extraction, API versus local deployment, prompt engineering, output validation and comparing closed- and open-source models.
2. **Google Cloud Video Analysis**
I integrated Google Cloud Storage and the Video Intelligence API to upload videos and extract duration, shot boundaries, labels, object tracks, confidence values and human-presence information.
3. **Feature Engineering**
I converted the raw GCP metadata into the main features: shot frequency, object entropy, interaction density and human presence ratio.
4. **Normalization and Scene Interpretation**
The extracted features were normalized and combined into a Scene Complexity Score. A rule-based classifier was then implemented to classify scenes as Calm, Dynamic, Dense or Static.
5. **LVLM Integration**
I implemented frame extraction and connected the pipeline to GLM vision models. The LVLM initially produced free-text summaries and was later extended to return structured metadata such as setting, entities, actions, emotions and interaction level.
6. **Prompt Engineering and Model Tuning**
Several prompt versions and two GLM models were tested. The interaction-level definitions were refined to improve recognition of subtle actions such as glances, one-sided communication and shared activities, also including tuning the feature stats using methods like mean, median, min, max and ranges.
7. **Pipeline Integration and Reliability**
The project was reorganized into reusable modules and the full pipeline was created to coordinate GCP analysis, frame extraction, LVLM inference, parsing, validation, feature calculation and JSON output generation. Retries, fallbacks and validation were added to handle overloaded APIs and malformed responses.
8. **Testing and Optimization**
The system was tested on increasing subsets of ACCEDE videos. Feature distributions, phase classifications, runtime, prompt accuracy and LVLM interaction predictions were evaluated and the normalization ranges and classification thresholds were adjusted accordingly.

- **4.5 Design Choices and Reasoning**

Throughout development, several design decisions were made to balance accuracy, interpretability, cost, reliability and implementation feasibility.

1. **Hybrid GCP and LVLM Architecture**

Reasoning: Google Cloud Video Intelligence provides reliable low-level metadata, while an LVLM provides higher-level semantic understanding.

Choice: I combined both layers so that deterministic visual features and semantic interpretation complement each other.

2. **Rule-Based Scene Classification**

Reasoning: The project did not have a large labelled dataset suitable for training a supervised classifier.

Choice: I used normalized features and configurable thresholds to classify scenes as Calm, Dynamic, Dense, or Static, while also storing the reasons for each decision.

3. **Interpretable Feature Engineering**

Reasoning: The system needed measurable features that could be clearly explained and tuned.

Choice: I selected shot frequency, object entropy, interaction density and human presence ratio because they represent pacing, visual diversity, scene busyness and human-centered content.

4. **API-Based LVLM Usage**

Reasoning: Running large open-source vision-language models locally required more computational resources than were available.

Choice: I used GLM models through an API and processed sampled video frames, allowing practical experimentation without local GPU infrastructure.

5. **Structured LVLM Output**

Reasoning: Free-text summaries are difficult to validate, compare and reuse programmatically.

Choice: I introduced versioned prompts that request structured JSON fields such as setting, entities, actions, emotion words, interaction level and supporting evidence.

6. **Modular Pipeline Design**

Reasoning: A single large script would be difficult to test, maintain and extend.

Choice: GCP analysis, feature engineering, normalization, classification, LVLM inference, parsing, validation and pipeline orchestration were separated into dedicated reusable and easily expandable modules.

7. **Separate Tuning Modes**

Reasoning: Re-running all APIs during every experiment would increase processing time and cost, as we wished to be as efficient as possible.

Choice: The pipeline supports full analysis, GCP-only tuning and LVLM-only tuning, allowing each layer to be tested independently.

- **4.6 Challenges and how they were handled**

Throughout the development process, I encountered several research, technical and design-related challenges. Below are the main issues I faced and the strategies I used to overcome them.

1. Defining the Project Direction

Challenge:

The project began from the foundation of the previous project, which did not succeed as intended. I needed to redefine the project's purpose, determine a more suitable research direction and avoid repeating the same structural mistakes, as well as repurpose the previous project's skeleton and ideas.

Solution:

I spent several months researching video analysis, competing solutions, current technologies and possible use cases.

With guidance from my lecturers and my supervisor, I refined the project into a research-oriented video-analysis tool that combines deterministic metadata with LVLM-based semantic interpretation.

2. Deciding Whether to Build a Frontend

Challenge:

Because many student projects include a website or application and because I had developed a frontend for the previous project, I initially assumed that this project also required one.

Solution:

After discussions with my supervisor, We concluded that the project should be treated as an R&D analysis tool rather than a consumer application.

Therefore, I prioritized the pipeline, experiments, evaluation and structured outputs instead of investing development time in a frontend that was unnecessary to my project.

3. Selecting the Appropriate Tools and Models

Challenge:

There were many possible APIs, LLMs and LVLMs, each with different capabilities, costs, hardware requirements, input formats and levels of accessibility.

Open-source models often required strong local GPU resources, while closed-source models depended on paid APIs.

Solution:

I compared multiple platforms and models, including ChatGPT, Gemini, Claude, DeepSeek, Perplexity, Grok, Qwen, LLaVA-related models, Google Cloud Video Intelligence and GLM vision models.

With guidance and collaboration with Dr. Shmuel Tyszberowicz and his Chinese student, I selected an API-based approach using Google Cloud Video Intelligence and GLM models, which matched the available hardware and the project's research goals (with the help of my friend's API key).

4. Combining Low-Level and Semantic Analysis

Challenge:

Google Cloud Video Intelligence provided reliable object tracks, labels and shot boundaries, but did not explain the semantic meaning of the scene.

The LVLM produced richer descriptions, but its outputs were interpretive and less deterministic.

Solution:

I designed a hybrid architecture in which GCP provides measurable visual metadata and the LVLM provides semantic interpretation. I kept these outputs separate but complementary, especially distinguishing GCP-based `interaction_density` from LVLM-based `interaction_level`.

5. Tuning the Narrative Phase Classifier

Challenge:

The initial normalization ranges and thresholds caused all 21 tested videos to be classified as Calm, which showed that several phase conditions were unreachable.

Solution:

I collected feature distributions, adjusted normalization ranges and updated the phase thresholds. The phase distribution improved from 21 Calm scenes to a mix of Calm, Dynamic, Dense and Static classifications (as its further detailed in the Markdown files in Github).

6. LVLM Interaction-Level Accuracy

Challenge:

The LVLM initially returned too many interaction-level values of 0 and failed to recognize subtle interactions such as glances, one-sided communication and shared activities.

Solution:

I refined the interaction scale, added clearer examples and rules to the structured prompt (while trying to keep it as general as possible), tested multiple prompt versions and compared GLM-4.6V-Flash with GLM-5V-Turbo. This improved the results, although the field remained interpretive rather than ground truth.

7. API Failures and Overload

Challenge:

During batch execution, the LVLM API sometimes returned overload errors, content-filter errors, or empty responses. GCS connections also occasionally failed.

Solution:

I added retry logic, fallbacks, validation checks, clearer error reporting and null-safe output handling. I also changed problematic prompt wording that triggered content filters and created a smoke-test script for verifying the LVLM pipeline independently.

8. Pipeline Efficiency and Code Duplication

Challenge:

The pipeline initially extracted frames more than once and contained repeated functions across several scripts, making the system less efficient and harder to maintain.

Solution:

I refactored the code so that frames were extracted once and reused, shared functions were centralized and the full and batch pipeline scripts were updated to avoid unnecessary processing. The final execution logic also skips videos that already contain valid results unless forced to rerun.

5. Implementation

- **5.1 Practical details**

of how the system or solution was built.

The system was developed as a modular Python pipeline, separating cloud-based video analysis, feature engineering, LVLM processing, validation, orchestration and output generation.

Google Cloud Video Analysis

I used Google Cloud Storage to upload local video files and Google Cloud Video Intelligence to analyse them.

The API extracts:

- video duration
- shot boundaries
- labels
- object tracks
- confidence scores
- person-presence intervals

The extracted metadata is processed and saved in VideoFeatures.json.

Feature Engineering and Algorithm Layer

The algorithm layer reads the GCP metadata and calculates 4 core features:

- shot_frequency
- object_entropy
- interaction_density
- human_presence_ratio

These values are normalized and combined into a weighted scene_complexity_score.

A rule-based classifier then assigns one of four narrative phases: Calm, Dynamic, Dense, or Static. The output also includes a score breakdown and textual reasons for the selected phase.

LVLM Analysis

I used GLM vision models through an API, initially glm-4.6v-flash and later glm-5v-turbo.

Representative frames are extracted from each video and sent with versioned prompts.

The LVLM produces:

- a free-text scene summary
- setting
- main entities
- actions
- emotion words
- interaction level
- interaction evidence.

The structured response is parsed, cleaned and validated before being added to the interpretation output.

Backend Pipeline

The backend is implemented as command-line Python pipeline modules.

It manages:

- video-path validation
- GCS upload
- GCP analysis
- frame extraction
- LVLM summary and structured inference
- response parsing
- feature validation
- normalization
- complexity calculation
- narrative phase classification
- JSON generation

The system supports full execution, GCP-only processing, GCP tuning, LVLM tuning, single-video runs and batch processing.

Retries, fallbacks and skip logic were added to reduce unnecessary API calls and handle temporary failures.

Data Organization

Each analysed video receives a dedicated directory under:

outputs/<video_id>/

The main files are:

VideoFeatures.json – GCP metadata and extracted numeric features.

VideoInterpretation.json – normalized features, complexity score, phase classification, LVLM summary and structured semantic metadata.

Configuration values such as model names, normalization ranges, weights, thresholds, frame settings and timeouts are centralized in configuration modules.

API credentials are loaded through environment variables.

- **5.2 Key modules**

data structures, software components and interfaces.

The system is divided into independent modules, each responsible for a specific stage of the analysis pipeline.

Key Software Modules

src/gcp/upload.py – Uploads local video files to Google Cloud Storage and returns the corresponding GCS URI.

src/gcp/analyze.py – Sends the uploaded video to Google Cloud Video Intelligence and retrieves shots, labels, object tracks and temporal metadata.

src/gcp/feature_engineering.py – Converts the raw API annotations into structured video metadata and saves VideoFeatures.json.

src/algorithm/features.py – Calculates the raw algorithm features, including shot frequency and interaction density.

src/algorithm/normalize.py – Normalizes the 4 core features according to configured ranges.

src/algorithm/complexity.py – Computes the weighted Scene Complexity Score and its feature contribution breakdown.

src/algorithm/phase.py – Applies rule-based thresholds to classify the scene as Calm, Dynamic, Dense, or Static.

src/algorithm/validate.py – Validates raw features, normalized values and structured outputs.

src/lvlm/client.py – Sends sampled frames and prompts to the GLM vision model, including retry and error-handling logic.

src/lvlm/prompts.py – Stores versioned summary and structured prompts.

src/lvlm/parse.py – Cleans the LVLM response, removes code fences, extracts JSON and parses the structured result.

src/pipeline/run_full_pipeline.py – Coordinates the complete processing of one video.

src/pipeline/run_batch_pipeline.py – Runs the pipeline on multiple videos and supports full, GCP tuning and LVLM tuning modes.

Data Structures

Python dictionaries and lists – Store API annotations, feature values, phase reasons, LVLM metadata and validation results.

JSON files – Used as the main persistent data format.

Frame collections – Store sampled video frames and timestamps used for LVLM inference.

Main Interfaces

Local video → **Google Cloud Storage** through the GCS upload client.

GCS URI → **Google Cloud Video Intelligence API** for deterministic video analysis.

Sampled frames + prompt → **GLM API** for semantic interpretation.

Pipeline modules → **JSON outputs** through:

- VideoFeatures.json
- VideoInterpretation.json

- **5.3 Challenges Encountered and How They Were Solved**

Duplicate Frame Extraction**Challenge:**

The LVLM workflow initially extracted the same video frames more than once for summary and structured analysis, increasing runtime and unnecessary processing.

Solution:

I refactored the pipeline so that frames are extracted once and reused for all LVLM requests.

Incomplete Pipeline Outputs**Challenge:**

In some cases, VideoFeatures.json was created successfully, but VideoInterpretation.json remained incomplete because the LVLM stage failed or returned invalid content.

Solution:

I added validation, fallback values and rerun logic so that incomplete or failed interpretation files are detected and processed again instead of being treated as valid results.

Empty LVLM Responses**Challenge:**

The GLM API occasionally returned empty content because the model's reasoning mode consumed the available token budget.

Solution:

I disabled the thinking mode for the project's LVLM calls and added explicit checks that raise a clear error when the response content is empty.

6. Experiments and Results

- **6.1 Experimental Setup**

Video Dataset

I used the ACCEDE dataset, which contains short video clips from Creative Commons films. The local collection contains 570 .mp4 files, while smaller subsets were used for feature tuning, prompt refinement, regression testing and final evaluation.

Feature Extraction and Models

Google Cloud Video Intelligence was used to extract shot boundaries, labels, object tracks, confidence values and human-presence information.
ffmpeg library was used for video probing and frame extraction, while glm-5v-turbo was used through the Z.ai API for scene summaries and structured semantic analysis.

Hardware Environment

Operating system: Windows 11 Pro 64-bit
Processor: Intel Core i5-1135G7 @ 2.40 GHz
Memory: 8 GB DDR4 RAM
Graphics: Intel Iris Xe integrated graphics
Storage: 476 GB Intel SSD

Because the machine does not include a dedicated high-performance GPU, I used cloud APIs instead of running large open-source LVLMS locally.

Software Environment

Python: 3.12.0 x64
ffmpeg / ffprobe: 7.1.1
Git: 2.44.0
Shell: Windows PowerShell 5.1
IDE: PyCharm
Pipeline interface: Python command line
External services: Google Cloud Storage, Google Cloud Video Intelligence and Z.ai's GLM API

Python Dependencies

The main pinned dependencies were: (as it is described in requirements.txt)

```
openai==2.25.0
ffmpeg-python==0.2.0
google-cloud-storage==3.8.0
google-cloud-videointelligence==2.18.0
google-api-core==2.29.0
requests==2.32.4
```

All required imports succeeded and all 38 offline unit tests passed in the verified Python 3.12 environment.

Evaluation Metrics

The system was evaluated using:

- expected versus predicted narrative phases
- expected versus predicted LVLM interaction levels
- exact and partial match rates
- phase-distribution changes after threshold tuning
- processing time for single-video and batch execution
- manual inspection of scene summaries, interaction evidence and structured outputs.

- **6.2 Results Table**
Quantitative Results

I evaluated the system in two main areas: numeric scene classification and LVLM interaction-level prediction.

Feature Statistics

The Week 5 analysis included 21 processed videos and showed the following raw feature distributions:

Feature	Minimum	Median	Mean	Maximum	Final Normalization Range
Shot frequency	0.0964	0.1920	0.2073	0.4660	0.0–1.0
Object entropy	1.5219	2.6981	2.5640	3.4601	0.0–4.0
Interaction density	0.2915	1.6302	1.8693	4.8857	0.0–5.0
Human presence ratio	0.4096	0.9742	0.9185	0.9960	0.0–1.0

These results showed that the tested clips were generally human-centered, with relatively low shot frequency and moderate object diversity and activity.

Narrative Phase Tuning

The original configuration classified all 21 videos as Calm.

After normalization and threshold tuning, all four narrative phases became reachable.

Tuning Stage	Calm Dynamic Dense Static			
Initial configuration	21	0	0	0
After normalization tuning	18	0	3	0
Final threshold configuration	15	1	2	3

The final distribution was more varied and realistic, although Calm remained the dominant class because it is used as the fallback for borderline cases.

(the 3rd tuning distribution was: 16 – calm, 2 dynamic, 1 dense, 2 static – which made the stats worsen so I reverted to the final configuration as seen above).

LVLM Interaction-Level Evaluation

The final LVLM evaluation used 40 ACCEDE clips with manually assigned expected interaction levels. The model produced an exact match for 25 of the 40 videos, resulting in an exact-match rate of 62.5%. Most remaining predictions differed from the expected result by one level and were therefore marked as partial matches. The full pipeline processed the 40 videos in approximately 28 minutes and 40 seconds.

Video Clip	Expected Level	Predicted Level	Result
ACCEDE09230 0	1		Partial
ACCEDE09231 1	1		Exact
ACCEDE09232 0	0		Exact
ACCEDE09233 1	1		Exact
ACCEDE09235 2	2		Exact
ACCEDE09237 3	2		Partial
ACCEDE09244 2	1		Partial
ACCEDE09257 2	2		Exact

Runtime Results

Google Cloud analysis for individual clips generally required approximately 18–33 seconds per video, depending on video length, network conditions and cloud response time.

The LVLM-only evaluation processed 21 videos in 5 minutes and 51 seconds, averaging approximately 16 seconds per video.

The complete pipeline - GCS upload, GCP analysis, algorithmic interpretation and LVLM inference - processed 40 videos in 28 minutes and 40 seconds, averaging approximately 43 seconds per video.

A GCS connection failure during testing required a retry, demonstrating the importance of the implemented retry and fallback mechanisms.

Qualitative Results

The LVLM consistently produced useful structured information, including:

- scene setting
- main visible entities
- actions and activities
- emotion-related words
- interaction level and evidence
- natural-language scene summary

The summaries were generally descriptive and aligned with the visible content. The interaction-level field improved after prompt tuning, particularly for visible communication and shared activities, but subtle or ambiguous interactions were still sometimes under- or overestimated. Overall, the experiments showed that the hybrid architecture successfully combines measurable GCP features with richer LVLM interpretation. The numeric classifier became more expressive after tuning, while the LVLM provided meaningful semantic metadata with moderate exact-match accuracy.

- **6.3 Analysis**

Effective Hybrid Analysis

The hybrid design was effective because each layer addressed a different limitation. Google Cloud Video Intelligence supplied consistent quantitative metadata about shots, objects and human presence, while the LVLM supplied contextual interpretation of settings, actions, emotions and interactions.

Together, these layers produced a more complete and explainable scene representation than either approach could provide independently.

Strength – Interpretability

The algorithmic layer is based on explicit features, normalization ranges, weighted scoring and rule-based phase thresholds. The system also stores a complexity breakdown, phase reasons and LVLM interaction evidence, making the outputs easier to inspect and explain.

Strength – Modular and Automated Pipeline

The full pipeline automates video upload, cloud analysis, feature engineering, LVLM inference, validation and JSON generation. Its modular design also allows GCP statistics and LVLM prompts to be tuned independently without rerunning every stage.

Weakness – Dependence on Human Judgment

Both narrative phase tuning and LVLM interaction-level evaluation require manual review. I assigned expected labels by inspecting the clips and compared them with the system outputs. This was necessary because the ACCEDE dataset does not provide ground-truth labels for the project's custom phase categories or interaction scale, but it also introduces subjectivity into the evaluation.

Weakness – LVLM Interaction Ambiguity

The LVLM performs well on clear interactions, but subtle communication such as brief glances, one-sided speech, or passive reactions can still be under- or overestimated.

The final interaction-level results should therefore be treated as semantic estimates rather than ground-truth measurements.

Weakness – External Service Dependence

The system relies on Google Cloud and the Z.ai API. Network conditions, service overload, rate limits, content filters and API availability can affect runtime and occasionally require retries or fallbacks.

Evaluation Method

Because there is no public benchmark that directly matches the project's custom Scene Complexity Score, narrative phases and interaction-level scale, I evaluated the system through manual annotation, expected-versus-predicted comparisons, exact and partial match scoring, feature-distribution analysis and repeated tuning experiments.

Future Potential

I plan to improve the feature engineering and algorithmic statistics derived from the GCP layer, especially the normalization ranges, phase thresholds and features used to distinguish borderline scenes.

This would allow the overall project accuracy to reflect both the deterministic algorithmic layer and the LVLM semantic layer, instead of relying mainly on LVLM interaction-level accuracy.

Further work could also introduce motion-aware features, larger manually reviewed evaluation sets and stronger methods for combining numeric and semantic evidence.

7. Discussions

- **Insights Gained**

The project showed that combining deterministic video metadata with LVLM-based semantic interpretation produces a more complete scene analysis than relying on either method alone. The GCP layer provides measurable information about pacing, object diversity, scene busyness and human presence, while the LVLM adds contextual understanding of entities, actions, emotions and visible interaction.

The experiments also showed that normalization ranges, classification thresholds, prompt wording and model selection have a strong effect on the final results.

Manual inspection remained necessary when evaluating narrative phases and interaction levels, especially in subtle or ambiguous scenes.

- **Limitations**

The narrative phase classifier still tends to over-predict Calm because it is the fallback category for scenes that do not clearly satisfy the other rules.

The current GCP-derived features may therefore be insufficient for distinguishing some borderline Static, Dynamic and Dense scenes.

The LVLM interaction level is interpretive rather than an objective measurement.

Brief glances, one-sided speech, passive reactions and weak shared activities may still be misclassified. In addition, the evaluation relies on manually assigned expected values, which introduces subjectivity.

The system also depends on external cloud services. Network conditions, API availability, content filters, rate limits and service overload can affect processing time and reliability.

- **Potential Improvements**

I plan to continue tuning the GCP and algorithmic layers by improving the feature definitions, normalization ranges, Scene Complexity Score and narrative phase thresholds.

A larger manually reviewed evaluation set should be created so that an accuracy score can be calculated for the algorithmic stage and combined with the LVLM evaluation, allowing the reported performance to represent the complete project rather than the LVLM fields alone.

I also plan to create an equivalent before-and-after tuning table for the GCP and algorithmic stage, similar to the LVLM prompt-update evaluation.

The evaluation results should be transferred from Markdown files into organized Excel tables to simplify filtering, comparison, statistical analysis and presentation.

Further research should examine current video-understanding methods, prompt-engineering practices, multimodal evaluation techniques and newly released LVLMs.

I may also test open-source LVLMs on a stronger desktop computer or through Afeka's VLab or another virtual machine, since these models may require substantially more GPU memory and processing power than my laptop provides.

Comparing local open-source models with the current cloud-based GLM approach could help evaluate accuracy, cost, privacy, runtime and reproducibility.

8. Conclusion and Future Work

- I successfully developed a modular video-analysis pipeline that combines Google Cloud Video Intelligence, feature engineering, rule-based scene classification and LVLM-based semantic interpretation.
- The system produces structured outputs describing scene complexity, narrative phase, visible entities, actions, emotions and interaction levels with videos as user input.
- The project demonstrated the value of combining deterministic video metadata with higher-level LVLM reasoning to create a more complete and explainable representation of short video scenes.
- Future work will focus on improving the GCP-derived features, normalization ranges, Scene Complexity Score and narrative phase thresholds, while expanding evaluation so that the project's overall accuracy reflects both the algorithmic and LVLM layers.
- Additional improvements may include organizing evaluation results in Excel, creating before-and-after tuning tables for the algorithmic stage, testing more recent LVLMs and experimenting with open-source models on a stronger desktop computer or through a virtual machine environment.

9. References

- **Paper:**
Link: <https://ieeexplore.ieee.org/document/8014905>
Deceiving Google's Cloud Video Intelligence API Built for Summarizing Videos
- **Paper:**
Link: <https://arxiv.org/abs/2407.01449>
ColPali: EFFICIENT DOCUMENT RETRIEVAL WITH VISION LANGUAGE MODELS
- **Videos Dataset:**
Link: <https://multimediaeval.github.io/2015-AffectiveTask>
Redi, M., Kofler, C., & Hauger, D. (2014). *ACCED: Affective Video Content Database*.
Multimedia Eval Workshop (MediaEval).

10. Credits

- I would like to express my sincere gratitude to **Dr. Shmuel Tyszberowicz** and his Chinese Student, **Zhengyuan Dai**, for their valuable assistance with the LVLM research.
Their guidance helped me better understand model selection, keyframe extraction, prompt engineering, API-based versus local deployment, output validation and useful research resources such as Hugging Face and model leaderboards (Ollama).
- I would also like to thank my dear friend **Lezeff** for providing access to the GLM Z.ai API key through his annual \$720 subscription, which made it possible for me to test and integrate the GLM vision models into the project.
(Welcomed to check his website - <https://ailang.tech/>)
- Finally, I would like to give special thanks to my supervisor, **Eyal**, for his continuous support, guidance and encouragement throughout the entire development process, from defining the project direction to completing the final implementation and research.